

Tarn

SDK Guide

Building apps on permanent, encrypted, user-owned data

June 2026

Contents

Tarn Client

- Quick start
- Schema
- Collections
- Connections
- Account key
- Passkeys
- Account + Session
- Advanced (escape hatches)
- App registration
- Examples
- TypeScript
- Error taxonomy
- Rate limiting
- Security model

Tarn Client

App-facing SDK for [Tarn](#) — permanent, encrypted, user-owned data on Arweave.

The SDK is schema-first: you declare collections and fields up front, and the client gives you typed CRUD per collection plus typed namespaces for connections, sharing, the account key, account management, and sessions. Apps never touch Arweave txids, content-encryption keys, share-log seq numbers, or HPKE handshakes — those stay inside the SDK.

Ships full TypeScript declarations. Works in browsers and Node.js 18+.

Quick start

```
import { TarnClient, defineSchema, TarnStorage } from 'tarn-client';

const schema = defineSchema({
  appId: 'your-app',
  version: 1,
  collections: {
    notes: {
      primaryKey: 'noteId',
      fields: {
        noteId: 'string',
        title: 'string',
        body: 'string?',
        priority: 'integer?',
      },
      shareable: true,
    },
  },
});

const tarn = await TarnClient.create({
  apiBase: 'http://localhost:8787',
  appId: 'your-app',
  schema,
  storage: TarnStorage.memory(),
});

await tarn.register('me@example.com', 'p@ssw0rd', { recoveryAcknowledged: true });
// The first argument is the username (any UTF-8 string the app chooses — many
// apps populate it with an email address; Tarn does not require email format).

await tarn.notes.create({ noteId: 'n1', title: 'Hello, Tarn' });
console.log(await tarn.notes.list());
```

That's the whole shape. Everything below is detail.

Schema

A schema declares what collections exist, what fields each collection has, and which collections can be shared. `defineSchema()` brands the result so the client only accepts validated schemas, and validates the shape eagerly at module load.

```
import { defineSchema } from 'tarn-client';

export const schema = defineSchema({
  appId: 'your-app',
  version: 1,
  collections: {
    notes: {
      primaryKey: 'noteId',
      fields: {
        noteId: 'string',
        title: 'string',
        body: 'string?',
        priority: { type: 'integer', required: false },
        updatedAt: 'date?',
        pinned: 'boolean?',
        status: { type: 'string', enum: ['draft', 'active', 'archived'] },
      },
      shareable: true,
    },
    settings: {
      primaryKey: 'key',
      fields: {
        key: 'string',
        value: 'json',
      },
      // shareable defaults to false – share()/listShared() are not available.
    },
  },
});
```

Field types: `string`, `number`, `integer`, `boolean`, `date`, `json`. Append `?` for optional (`'string?'`), or use the long form `{ type, required, default, enum }`.

`date` **fields round-trip as** `Date`. A `date` field accepts either a `Date` or an ISO-8601 string on write (both are coerced to a `Date` and validated). On the wire the value is stored as an ISO string (JSON has no `Date` type), and the SDK **re-hydrates it back to a** `Date` **on read** — `get()`, `list()`, `getEntriesSince()`, and `listShared()` all return `Date` objects for `date` fields, matching the declared TypeScript type. A stored value that can't be parsed as a date (corrupt/legacy data) is left untouched rather than throwing. If you want a numeric timestamp instead of a `Date`, declare the field as `number` (ms-epoch) — that's stored and returned verbatim with no coercion.

`default` **values are validated at** `defineSchema()` **time**. A long-form field's `default` is checked against the field's own type and enum when the schema is defined, not lazily at the first `create()` that omits the field — so a malformed default (wrong type, value outside the enum,

unparseable date) throws `TarnSchemaError` immediately, pointing at the schema where the mistake lives.

Reserved names: `cred`, `connection`, `share-log-state`, `share-inbox`, `recovery-factor`, `app-config`, `app-schema`. Declaring a collection with a reserved name throws at `defineSchema()`.

Sharing: only collections with `shareable: true` get `share/shareWithAll/unshare/listShared` methods. The schema is the authoritative answer to "is this thing shareable" — apps can't bypass it.

Validation: `tarn.<collection>.create()` and `update()` validate against the schema synchronously, before any network or crypto work. Missing required fields, wrong types, unknown fields, or enum violations throw `TarnSchemaError`.

Schema versioning and field evolution

Schemas carry a numeric `version`. Bumping it republishes the schema (via `tools/publish-schema.mjs`) under a new `v=` Arweave tag so the recovery client can pick the version that matches each record's `SchemaV` tag.

Read-side version dispatch. Every record carries the `SchemaV` it was written under, and the SDK reads it on every fetch (Tarn #37). The contract:

- **Older record, newer client** → read normally (the additive-change rows below), or transformed by a declared migrator (see "Inline forward-migrators").
- **Same version** → normal read.
- **Newer record, older client** (a record written by an app version that's ahead of the one running) → the SDK will **not** feed it to the strict validator (that would silently corrupt it).
`list()` and `getEntriesSince()` **skip it with a console warning** so one newer record can't break a whole read; single-record `get()` / `update()` **throw** `TarnSchemaVersionError` (catch it and prompt the user to upgrade the app). `delete()` still works on such a record.
- **No `SchemaV` tag** (legacy data) → treated as the oldest version; never crashes.

So, today, **only the backward-compatible (additive) changes in the table below are safe by default**. A breaking change requires either a default/deprecation (per the table) or an inline forward-migrator.

*"Additive" means added-**OPTIONAL** (or added-required-WITH-a-default). The read path does **not** re-run strict validation (that would risk throwing on legitimate legacy data), so an older record that's missing a newly-added required, no-default, no-migrator field is returned **as-is** rather than rejected — it would otherwise violate the record's declared TypeScript type. To make this visible, the SDK emits a `console.warn` on read when it detects an older record missing such a field (Tarn #57). Treat that warning as "fix your schema": add a `default`, or declare a `migrations[v]` that fills the field. Reads never throw or drop the record on this case — the warning is the contract enforcement.*

Three rules cover the common cases:

Change	Backward-compat?	What you do
Add a new optional field	yes	Bump <code>version</code> , republish. Older records read with the field absent; new writes include it.
Add a new required field with a default	yes	Bump <code>version</code> , republish. The default fills in for older records on read.
Add a new required field without a default	no	Older records fail validation on read. Either supply a default, or migrate (see below).
Remove a field	no	Older records still carry the field on disk; the strict validator rejects unknown fields on read. Either keep the field declared as deprecated (still accepted, no longer used), or migrate.
Rename a field	no	Same as remove + add. Migrate, don't rename in place.
Change a field's type	no	Always a breaking change. Migrate.
Tighten an enum (drop a value)	no	Older records carrying the dropped value fail. Don't drop; deprecate.
Loosen an enum (add a value)	yes	Older records still satisfy the union.

Inline forward-migrators (the read-time seam). `defineSchema()` accepts an optional `migrations` map. `migrations[v]` transforms a record written under version `v` into the shape of version `v+1` . The SDK chains these in memory on read for any record older than the current `version` — so an older record can be reshaped on the fly without rewriting it on Arweave. The map is validated at `defineSchema()` time (keys must be positive integers strictly below `version`).

Two shapes are accepted:

```

const schema = defineSchema({
  appId: 'myapp',
  version: 2,
  collections: { profiles: { /* ... */ }, notes: { /* ... */ } },

  // RECOMMENDED – collection-scoped: each migrator runs ONLY against its own
  // collection's records.
  migrations: {
    profiles: {
      // v1 profiles had `name`; v2 splits it. Applied only to profile entries.
      1: (old) => ({ ...old, displayName: old.name, name: undefined }),
    },
    notes: {
      1: (old) => ({ ...old, body: old.text, text: undefined }),
    },
  },
});

// LEGACY / single-collection – flat: the SAME migrator runs against EVERY
// collection's records. Only safe for a single-collection schema, or migrators
// written to no-op on records that aren't theirs. Multi-collection apps should
// use the scoped shape above (Tarn #55).
migrations: {
  1: (old) => ({ ...old, displayName: old.name, name: undefined }),
}

```

The SDK disambiguates the two by their top-level keys: numeric keys = flat; collection-name keys = scoped. In the scoped shape, a key naming an undeclared collection throws at `defineSchema()`. **If a step in the chain has no migrator while the map is otherwise populated** (e.g. you declared `1` and `3` but forgot `2`), the SDK assumes that step was additive and skips it — but emits a `console.warn` so an accidental gap is visible (Tarn #58c). If the step was *not* additive, declare a migrator for it.

This is the lightweight, no-rewrite option — it does not persist anything, so it costs zero Arweave writes. Use it for shape changes you can compute purely from the old record.

One-shot rewrite migration. When you'd rather rewrite records permanently (or the transform isn't pure), Tarn doesn't ship a `migrate()` helper for the walk itself — apps write their own loop because the right semantics (one-shot vs. lazy, error handling, partial-failure recovery) are app-specific.

Cost matters. Every `update()` is a new permanent Arweave entry, paid via Turbo from the app's funder. Records aren't mutated in place — chained entries append a new version each time, and old versions stay on Arweave forever. A one-shot migration of N records on K users is $N \times K$ permanent paid writes. Small entries currently fall under Turbo's free tier, but that's pricing policy, not a guarantee.

Prefer schema changes that don't require migration. The first three rows of the table above — additive optional fields, additive required fields with defaults, loosened enums — cost zero writes. Reach for them first. Deprecate fields instead of removing them. Loosen enums instead of tightening. Most schema evolution can be designed to avoid migration entirely if you plan

additively from the start.

When migration is unavoidable, the pattern is straightforward:

```
// One-shot migration: read all, transform, re-write under the new schema.
// Each update is a new permanent Arweave entry – budget accordingly.
const all = await tarn.notes.list();
for (const note of all) {
  if (note.body == null) {
    await tarn.notes.update(note.noteId, { body: '' });
  }
}
```

Validation runs on `update()`, so the migration loop fails fast if the transform is wrong. Re-running is safe — `update()` is idempotent on identical inputs.

Forward-looking — relaxed validation. The current validator is strict by design: unknown fields throw. A future option (`onUnknownField: 'strip' | 'preserve' | 'error'`) will let apps opt into laxer behaviour for upgrade paths — read with extra fields, log a warning, drop them on the next write. Default stays `'error'` so the schema-first commitment isn't watered down. Not shipped yet; track the issue if it matters for your migration plan.

Collections

Each `shareable: true` collection on the schema becomes a typed namespace on the client.

```
// Create. Validates against the schema. Returns the validated record.
await tarn.notes.create({ noteId: 'n1', title: 'Quarterly review', body: 'Pull metrics'
  });

// Retry-safe create. Pass a STABLE idempotencyKey (derived from the record's
// persistent identity, e.g. its primaryKey) to make a logical write safe to
// re-attempt. A retry that sends the same key – an offline-replay path
// re-running a queued op after a crash, or a requeue after a lost response –
// hits the API's 24h dedup and returns the original result instead of writing
// a second entry. Omit it for the default (each call is treated as distinct).
await tarn.notes.create(
  { noteId: 'n1', title: 'Quarterly review', body: 'Pull metrics' },
  { idempotencyKey: 'notes:n1' },
);

// Get one. Returns null if no record matches.
const note = await tarn.notes.get('n1');

// List all live records.
const all = await tarn.notes.list();

// Partial update – SDK reads current, merges patch, writes a new entry.
await tarn.notes.update('n1', { priority: 5 });

// Tombstone.
await tarn.notes.delete('n1');
```

```
// Bulk create. Up to 25 records per call, validated as a batch.
// One rate-limit hit; every record stamped with its Eid + SchemaV tags.
await tarn.notes.batchCreate([
  { noteId: 'n2', title: 'First note', body: '...' },
  { noteId: 'n3', title: 'Second note', body: '...' },
]);
```

Records are addressed by **primary key**, never by Arweave txid. The SDK maps primary keys onto the protocol's `Eid` tag so reads converge across devices.

`create()` **on an existing primary key is last-write-wins by default**. Because a record's `Eid` is derived from its primary key, a second `create()` for the same key writes a new entry that *supersedes* the prior one (the old version stays on Arweave but is no longer the live record) — it does **not** error. This is intentional and matches `update()`'s `Eid` behavior. If a duplicate create signals a bug (e.g. you mint a fresh id that must be unique), pass `failIfExists` to reject it instead:

```
// Throws TarnCollectionError if a live record with this primary key exists.
await tarn.notes.create({ noteId: 'n1', title: 'New' }, { failIfExists: true });
```

`failIfExists` costs one extra read (a narrow `Eid` lookup) before the write, and is **best-effort, not atomic** — two truly-concurrent creates of the same new key can still both pass the check (last-write-wins then reconciles them via the shared `Eid`). For retry-safety of the *same* logical create, prefer `idempotencyKey` over `failIfExists`. `batchCreate` additionally rejects a batch up front if two items in it share a primary key (an in-batch collision would silently collapse to one record).

`update()` is **partial-merge**. Pass only what's changing; the SDK reads the current record from Arweave, merges, re-validates as a full record, and writes a chained entry. Full-replace is `update(id, { ...current, ...patch })` if you ever want it.

The primary key is immutable. A record's identity (`Eid`) is derived from `hash(appId, collection, primaryKey)`, so changing the primary key would mint a brand-new `Eid`, orphan the original record's history, and silently fork the logical record into two. `update()` enforces this: if your patch includes the primary-key field with a value that differs from the existing record's, the call throws a `TarnCollectionError` and nothing is written. Including the primary-key field with its *current* value is a harmless no-op — it's stripped before the merge. If you genuinely need to "rename" a record's key, model it as a delete of the old key plus a create of the new one.

Clearing fields. To remove a field from an existing record, pass its name via the `unset` option:

```
// Clear `dateRead` while also updating the title — both happen in one write.
await tarn.books.update('b1', { title: 'A New Title' }, { unset: ['dateRead'] });
```

The listed keys are deleted from the merged record *after* the patch, then the result is re-validated. Notes:

- Unsetting a field that wasn't present is a no-op.
- If the same field is in both `patch` and `unset`, the `unset` wins (delete-after-merge).

- Unsetting a required field throws the standard "required field missing" validation error — required fields can't be cleared.
- Unsetting an **optional field that has a default** truly clears it — it does *not* silently revert to the default. (The re-validation step knows the field was deliberately unset and skips default application for it.)
- Reading the record back via `get()` returns a record where the cleared key is truly absent (not `null` or `undefined`).

Bulk imports. `batchCreate` writes up to 25 records in one request and counts as a single rate-limit hit. Every record is validated against the schema before any wire call — if any record fails, nothing is written and the thrown `TarnCollectionError` lists the failing indexes:

```
// Migration: import 18 books from an exported library in one shot.
// 1 rate-limit hit + 1 round trip – vs 18 of each via single create().
await tarn.books.batchCreate([
  { bookId: 'b-001', title: 'The Idiot',          status: 'reading' },
  { bookId: 'b-002', title: 'Mountains of My Life', status: 'finished' },
  // ...up to 25 per batch
]);
```

This is the right path for bulk imports and migrations. Records written through it carry the same Eid + SchemaV tags as single `create()` calls, so they surface normally through `get`, `list`, and `getEntriesSince`. Larger imports: chunk client-side and pace at ~1 batch per 36s to stay under the 100 writes/hour/account limit. The schema-less escape hatch `tarn.advanced.entries.batchCreate` still exists for app-internal types that aren't on the schema — see the [Advanced](#) section for the trade-offs.

Typed reads drop orphan entries; untyped reads return them

Typed collection reads — `tarn.<collection>.list()` and `tarn.<collection>.getEntriesSince()` — are **Eid-addressed**. A record is identified by the Eid tag the SDK derives from its primary key, and every well-formed `create` / `update` / `batchCreate` through the typed surface stamps that tag. An entry on the wire that has **no Eid tag** ("orphan" — a legacy write, or a record written through the untyped escape hatch without per-item Eids) is by construction not something a typed collection produced, so the typed layer **drops it and logs a** `console.warn ( Collection '<name>': dropping orphan delta event... / skipping malformed entry... )`. Entries written under a *newer* SchemaV than the client knows are likewise skipped-with-warning, so one record from a future client can't break a whole-collection read.

The untyped layer makes the opposite choice. `tarn.advanced.entries.*` is a pure pass-through: a raw read of a type returns **every** entry the server has, orphans included — it has no Eid contract to enforce. So the same data can appear "missing" through `tarn.books.list()` yet present through the advanced surface. This is by design, not a bug: the typed surface trades completeness for a clean per-record identity, and the untyped surface is the escape hatch when you need to see (or repair) orphans. If a typed read is missing records you know were written, suspect orphans — read the type through `tarn.advanced.entries` to confirm, and migrate those writes to the typed surface (or re-stamp their Eids) to make them surface normally.

Sharing (when `shareable: true`)

```
const friends = await tarn.connections.list();

// Share one record with one friend.
await tarn.notes.share(friends[0], 'n1');

// Share with all connections (skips muted). Returns counts and per-conn failures.
const result = await tarn.notes.shareWithAll('n1');
// { ok: 3, failed: [{ connection, error }] }

// Revoke from one friend.
await tarn.notes.unshare(friends[0], 'n1');

// Read what a friend has shared with us under THIS collection.
const theirs = await tarn.notes.listShared(friends[0]);
```

Calling `share()` on a non-`shareable` collection throws — the schema is the gate.

Connections

`tarn.connections.*` covers the full lifecycle of friend connections.

```
// Username-based handshake (requires recipient to be a discoverable Tarn user).
await tarn.connections.invite('alice@example.com');

// Recipient lists incoming requests via the advanced surface, then accepts:
await tarn.connections.accept(requestNonce, { label: 'Alice – work team' });

// Listing.
const conns = await tarn.connections.list();
// [{ share_pub, signing_pub, label?, muted? }, ...]

// Mute / unmute. Mute is per-side, invisible to the other party.
await tarn.connections.mute(conns[0]);
await tarn.connections.unmute(conns[0]);
const muted = await tarn.connections.isMuted(conns[0]);

// Set or change a label after the fact.
await tarn.connections.setLabel(conns[0], 'Alice');

// Remove the connection. They stay around in your share-log history but
// new shares stop flowing.
await tarn.connections.remove(conns[0]);
```

Invite tokens — link / QR flow

For consumer apps where the inviter doesn't know the recipient's username (or the recipient isn't a Tarn user yet):

```
// Inviter – generate a single-use, time-limited link.
const { token_id, invite_url, expires_at } = await tarn.connections.createInvite({
  label:      'Maya', // local-only: labels the connection that forms when this
```

```

    redeems
    expiry_days: 7,          // default 7, server max 30
    recipient_metadata: { display_name: 'Maya' }, // app-defined, shown to the recipient
  });
// invite_url is something like:
// https://app.example.com/invite/<token_id>#<base64url payload_key>
// Share it via QR / messenger / email / etc.

// Recipient – extract token_id from path, payload_key from URL fragment.
const tokenId = new URL(location.href).pathname.split('/').pop();
const payloadKey = location.hash.slice(1);

// Pre-redemption landing page: non-consuming peek.
const preview = await tarn.connections.previewInvite(tokenId, payloadKey);
// preview.recipient_metadata → { display_name: 'Maya' } – render "Maya invited you".
// null preview means expired / already used / not found / wrong key.

await tarn.connections.redeemInvite(tokenId, payloadKey);
// The inviter's SDK auto-accepts the resulting connection request on next poll.

```

The URL fragment (#) is never transmitted to the API — the payload key stays on the recipient's device. Tarn stores opaque ciphertext keyed on `token_id` and never sees the inviter's identity.

`label` is local: it's stored only in the inviter's encrypted `tarn-issued-invites-v1` record, surfaced in `listIssuedInvites()`, and used to seed `Connection.label` when the matching redemption auto-accepts. The recipient never sees it.

`recipient_metadata` is the recipient-facing counterpart: an app-defined object (≤ 2048 bytes serialized) encrypted into the invite payload and returned — decrypted — by both `previewInvite()` and `redeemInvite()`. Tarn never interprets it and the server never sees it. Two caveats: anyone holding the full invite URL can decrypt it, so put no secrets in it; and it's inviter-asserted, not Tarn-verified, so render it as context ("Maya invited you") rather than authenticated identity.

Options objects across the connections surface are strict: an unknown key (say, `display_name` where the SDK expects `label` or `recipient_metadata`) throws immediately rather than being silently ignored.

Account key

Tarn issues every account a 24-word BIP39 account key at signup. The account key is a parallel access path to the user's data — independent of the password. Apps **must** surface the account key to the user during registration; passing `recoveryAcknowledged: true` is the SDK's way of forcing the conversation.

```

const reg = await tarn.register('me@example.com', 'p@ssw0rd', {
  // First arg is the username. See "Authentication and the username field" below.
  recoveryAcknowledged: true,
  // storeAccountKey defaults to true (Model B). Pass false for strict
  // zero-knowledge "no backup stored" registration (Model A).
});

```

```

    storeAccountKey: true,
  });
  // reg.accountKey          - 24-word string
  // Hand it to the user immediately. Do NOT persist it.

```

Kit format and delivery are the app's job. Tarn returns the 24-word account-key string and nothing else — no PDF, no rendered bytes. Apps render their own kit (downloadable PDF, printable HTML, clipboard copy, whatever fits) and decide how to surface it to the user. There is no Tarn endpoint that handles plaintext account-key material, even ephemerally; the SDK does not ship an in-bundle PDF renderer either, so apps stay in full control of branding and layout. If the application wants to email the kit, it must operate the transport itself — Tarn will not host a forwarder, since routing account-key material through Tarn-operated infrastructure would weaken the zero-knowledge guarantee that applies to everything else in the protocol.

Storage models — Model A vs Model B

Apps pick the storage posture at registration via the `storeAccountKey` option:

- `storeAccountKey: true` **(default, Model B)**. The SDK encrypts the account key under the user's gen-1 DEK with AAD `"tarn-wrapped-account-key-v1"` and ships the ciphertext to Tarn. Logged-in users can later retrieve and view the account key from app settings via `tarn.accountKey.view()`. The wrap is opaque to Tarn; only a user who supplies their password can decrypt it.
- `storeAccountKey: false` **(Model A)**. The wrap is omitted entirely. Tarn never holds any form of the account key. Lose the password AND the account key → data is permanently inaccessible (even Tarn cannot help). This is the strict zero-knowledge posture.

Reading the storage state at runtime:

```

// After login or register completes:
tarn.accountKey.isStored(); // true (Model B) | false (Model A) | null (no auth round trip yet)

```

Viewing the account key (Model B)

```

const { accountKey } = await tarn.accountKey.view({ password:
  'freshly-re-entered-password' });
// 24-word string. Surface it briefly; do not persist.

```

The flow:

1. The SDK runs a step-up auth dance against the API: fresh nonce → sign with credential signing key derived from the re-entered password → exchange for a single-use 60-second token.
2. The SDK fetches the encrypted wrap with both the session JWT and the step-up token.
3. The SDK decrypts the wrap client-side and runs a wrap-pinning check (re-derives the `recovery_lookup_key` from the decrypted phrase and compares to the server-stored value). On mismatch, `view()` throws `AccountKeyPinningError` — surface this distinctly from a "wrong password" or "network error", since it's a security warning.

Failure modes:

- **Wrong password** — step-up authentication fails. The SDK throws an `Error` whose message contains `"step-up"` or `"challenge"` .
- **Model A account** — the server returns 404. The SDK throws an `Error` whose message contains `"no_account_key_stored"` . Render the appropriate "no backup stored" UI.
- **Pinning mismatch** — the SDK throws `AccountKeyPinningError` . Treat as a security warning and avoid surfacing the (would-be) phrase.
- **Network / decryption failure** — propagated as a regular `Error` .

Toggling storage at runtime

Users can switch their storage posture at any time after registration. Both methods drive a step-up auth dance internally — the caller passes the freshly-entered password.

```
// Model A → Model B. Caller must also supply the user's existing
// account key (the SDK does not retain it past register / view / rotate).
// Apps prompt the user to type it, validate via validateAccountKey(),
// then pass it here.
await tarn.accountKey.enableKeyStorage({
  password: 'freshly-re-entered-password',
  accountKey: '24 words ...',
});
```

```
// Model B → Model A. Just the password – no phrase needed.
await tarn.accountKey.disableKeyStorage({ password: 'freshly-re-entered-password' });
```

`enableKeyStorage` runs a wrap-pinning check before submitting: it derives `recovery_lookup_key` from the supplied phrase and compares to the value cached on the client. If the user types a valid 24-word phrase that doesn't actually belong to this account, the SDK throws `AccountKeyPinningError` BEFORE any server round trip. The pin check requires the SDK to have learned `recovery_lookup_key` (set at register / recoverAccount); on a login-only session the value isn't available and the pin check is skipped — the server is the only line of defense in that case, which is fine.

`disableKeyStorage` is idempotent: calling it on an already-Model-A account returns `{ stored: false, alreadyDisabled: true }` instead of throwing. UI code can treat both responses identically.

Failure modes for both:

- **Wrong password** — step-up fails. Error message contains `"step-up auth failed"` .
- **Pin-check failure** (`enableKeyStorage` only) — `AccountKeyPinningError` .
- **Network / D1 failure** — propagated as a regular `Error` .

Rotating the account key

When the user suspects their existing account key has been compromised (or wants to evict it as a routine hygiene step), `rotate()` generates a fresh one, re-wraps the entire DEK chain under the new recovery factor, and atomically publishes the bundle. The OLD account key stops working for `recoverAccount` immediately after this returns.

```
const { accountKey: newPhrase } = await tarn.accountKey.rotate({
  password: 'current-password',
});
// Surface newPhrase to the user briefly (downloadable kit, printable
// page, etc.). The SDK does not retain it.
```

Notes:

- The user's password and username are NOT changed by rotation — only the account-key half of the recovery factor.
- The recovery salt is regenerated as part of the new envelope (clean reset of the recovery state).
- If the account is in Model B, the new account key is also stored (re-wrapped under the existing gen-1 DEK). Model A stays Model A.
- Pre-rotation data remains decryptable — the DEK chain itself is unchanged; only the wrappings rotated.
- Apps can also pass `rotatePhrase: true` to `tarn.recoverAccount()` to bundle a rotation into a forgot-password flow (see below).

Like `viewAccountKey` / `enableKeyStorage` / `disableKeyStorage`, `rotate` runs a step-up dance against `/auth/step-up` before submitting the rotation — the rotate endpoint requires both a session JWT and a fresh step-up token (Phase 4.1, mirrors the other privileged account-key endpoints). The SDK handles step-up internally; the caller still just passes `{ password }`.

Failure modes:

- **Wrong password** — local credential mismatch tripwire. Error message contains `"wrong password"`.
- **Conflict** — `409` from the API if the new `recovery_lookup_key` collides with another account (astronomically unlikely; retry yields a fresh key). Error message contains `"conflict"`.
- **Network / D1 failure** — propagated as a regular `Error`. The D1 update is atomic, so partial state cannot occur; safe to retry.

Account recovery (when the password is lost)

Account recovery itself goes through the top-level `tarn.recoverAccount()` (auth lifecycle):

```
await tarn.recoverAccount({
  phrase:      '24 words ...',
  newUsername: 'me@example.com',
  newPassword: 'fresh-password',
});
```

```
});
// New credentials re-wrap the existing data keys; all pre-recovery data is decryptable.
```

Optional `{ rotatePhrase: true }` . Pass this to bundle an account-key rotation into the recovery flow — useful when the user is recovering BECAUSE they suspect the phrase itself has been compromised (e.g. "I think someone has my recovery sheet"):

```
const result = await tarn.recoverAccount({
  phrase:      '24 words ...',
  newUsername: 'me@example.com',
  newPassword: 'fresh-password',
  rotatePhrase: true,
});
// result.accountKey contains the NEW phrase; surface it to the user
// once and let them save it. The OLD phrase no longer authenticates.
```

When `rotatePhrase` is omitted (default `false`), recovery preserves the existing phrase — exactly as documented in the protocol. The result has no `accountKey` field. If the inline rotation fails after a successful recovery, the SDK throws a partial-success error pointing at `tarn.accountKey.rotate()` for retry.

Authentication and the username field

Tarn does not validate the username's format. It's a UTF-8 string used as a KDF salt input (after `trim().toLowerCase()` normalization) and as a public user-lookup key for connection bootstrap. Apps choose whether to enforce email-shape, handle-shape, phone-shape, or anything else; the SDK neutrally calls the parameter `username` . Many apps will populate it with an email address — that's fine and expected — but Tarn does not require this.

Passkeys

Passkeys are an opt-in third independent factor for both encryption (the DEK chain gains a per-passkey wrapping derived via the WebAuthn PRF extension) and authentication (the assertion proves possession without prior session state). A user can register zero, one, or many passkeys per account; each one independently unwraps the data and each one can log in.

Passkeys require WebAuthn + the PRF extension. Coverage in mid-2026: Chrome 132+, Safari 18+, recent Edge. Firefox is still patchy. Apps **must** feature-detect before surfacing the affordance.

```
// Feature-detect first – render the UI only when this returns true.
if (await tarn.passkeys.isSupported()) {
  // Register a new passkey for the logged-in account. Triggers the platform
  // authenticator prompt (Touch ID / Face ID / Windows Hello / security key).
  // Pass a label so the user can recognize the device in their settings later.
  const { credentialId } = await tarn.passkeys.register({
    deviceLabel: 'Brian\'s iPhone',
  });

  // List registered passkeys (Settings UI).
  const all = await tarn.passkeys.list();
```

```

// [{ credentialId, deviceLabel, createdAt, lastUsedAt, stale }, ...]
// `stale: true` means the credential has no wrapping at the latest gen
// – typically because changeCredentials ran without a re-tap. Surface
// a "Refresh" affordance on stale entries (see "What happens when you
// change your password" below for the repair flow).

// Remove a passkey. Step-up gated – caller passes the freshly-typed password.
await tarn.passkeys.remove({
  credentialId: all[0].credentialId,
  password: 'the-password',
});
}

// Authenticate with a passkey instead of password. Returns the same logged-in
// state as `tarn.login()`; subsequent collection / sharing / settings calls
// work normally.
await tarn.authenticateWithPasskey({
  deviceLabel: 'Brian\'s iPhone',
  // Optional: handler for the stale-credential path. The handler MUST
  // return both username and password – passkey-only sessions don't
  // have either cached on the client. See "What happens when you change
  // your password" below for the repair flow.
  stalePasskeyHandler: async () => ({
    username: await app.promptForUsername(),
    password: await app.promptForPassword(),
  }),
});

```

What passkey support gets you. A registered passkey lets the user (1) log in without typing a password — the platform authenticator's biometric prompt is enough — and (2) decrypt all existing data without holding the password. The DEK chain gains a `passkey_prf` wrapping per gen at registration; subsequent reads through that passkey unwrap normally.

The passkey-only session model. A passkey-authenticated session has a different shape from a password-authenticated one: the client holds the JWT and the unwrapped DEK chain, but NOT the master_key-derived state (username, signing keypair, sharing keypair, credential-encryption key). That is enough to read, create, update, and delete entries — the JWT carries the auth context — and the session **persists and resumes** across reloads exactly like a password session does (`tarn.session.isLoggedIn()` returns `true` , the persisted blob round-trips, and the resumed client can keep reading and writing). What it cannot do is anything that needs the master_key: `changeCredentials` , `viewAccountKey` , `rotateAccountKey` , `registerPasskey` , `removePasskey` , and the sharing/invite handshake methods all throw a clear "requires a password-authenticated session" error. To perform any of those, the user signs in with username + password first.

What passkey support does NOT change. Adding a passkey does not weaken the password or account-key paths — the existing wrappings stay in place. Removing a passkey strips its wrappings without touching the others. `rotateAccountKey` and `recoverAccount` do not mint a new generation, so passkey unwrappability survives both unchanged.

Fallback. If `tarn.passkeys.isSupported()` returns false (Firefox, Chrome on a Linux box without a platform authenticator, etc.), apps should not surface the passkey affordance and should fall back to password-only auth. The SDK refuses to register a passkey on a device without PRF (the wrap would be unrecoverable), so a hopeful "let's just try" is not safe.

Multi-device. Apple iCloud Keychain and Google Password Manager sync passkeys across the user's devices; in those cases, registering on one device makes the credential available on all. For non-syncing authenticators (Windows Hello, hardware security keys), the user must register one per device they want to log in from.

Passkey session capabilities

The contract is "downstream symmetry, asymmetric initial auth". After the user is authenticated — whether by password or by passkey — the SDK surface is the same EXCEPT for a small, documented set of operations that genuinely require master_key-derived state (signing keypair, sharing keypair, credential-encryption key, username, credential-lookup key). Passkey-only sessions never derive that material; calling one of those operations on a passkey-only session throws `TarnPasskeyOnlyError` (exported from the package) so apps can render a clear "sign in with your password to do this" affordance.

Works identically on both auth methods (no special-casing required in apps):

- Entry CRUD on every collection: `create` , `get` , `update` , `delete` , `list` , `batchCreate`
- Delta sync: `getEntriesSince` , `getEntryByEid`
- Advanced entries surface: `tarn.advanced.entries.*`
- Blob fetch and shared-blob decryption
- Session lifecycle: `tarn.session.isLoggedIn()` , `serializeSession` , `TarnClient.resumeSession` , server-side session listing/revocation
- Reading shared content the recipient already holds the per-content `cek` for (the JWT carries the bearer; no per-pair derivation needed for the reader side of an already-resolved share)
- Passkey management on a session that's already authenticated by passkey:
`tarn.passkeys.list` , `tarn.passkeys.isSupported` , `authenticateWithPasskey`
(including stale-credential repair via `stalePasskeyHandler`)

Throws `TarnPasskeyOnlyError` **on a passkey-only session** (sign in with username + password first):

- Credential and account-key management — `changeCredentials` , `viewAccountKey` , `rotateAccountKey` , `enableKeyStorage` , `disableKeyStorage` , `tarn.passkeys.register` , `tarn.passkeys.remove`
- Connection handshake — `sendConnectionRequest` , `acceptConnectionRequest` , `createInviteToken` , `redeemInviteToken` , `listIncomingRequests`

- Share-log writes — `Collection.share` , `Collection.shareWithAll` , `Collection.unshare` , the underlying `shareContent` / `updateShareContent` / `unshareContent` / `snapshotShareLog` on the protocol layer, plus `removeConnection` and `revokeContentFromConnections`
- Share-log reads (pair-keyed) — `Collection.listShared` , the underlying `readShareLog` / `syncShareLog` . These derive a per-connection pair key from the user's `sharing_priv` , which passkey-only sessions don't carry.

Apps that need any of the asymmetric operations should catch `TarnPasskeyOnlyError` and route the user through a password sign-in (`tarn.login(username, password)`) before retrying. The error message contains "requires a password-authenticated session" so a generic message-substring check works too, but the typed error is the stable contract.

```
import { TarnPasskeyOnlyError } from '@tarn/sdk';

try {
  await tarn.books.share(connection, bookId);
} catch (err) {
  if (err instanceof TarnPasskeyOnlyError) {
    // Surface "Sharing requires your password – sign in to continue."
    await app.promptForPasswordSignIn();
    await tarn.books.share(connection, bookId); // retry after password auth
  } else {
    throw err;
  }
}
```

What happens when you change your password

`changeCredentials` mints a new generation of the data key. For password and account-key factors that's fine — the SDK has both KEKs in hand and re-wraps the new gen automatically. For passkeys, the SDK does NOT have the PRF output (it never persists it), so the new gen would ship without any passkey wrapping unless the user re-taps each registered authenticator at change time. Without that, passkey-only sessions could read pre-change data but not anything written under the new gen — a real break in the passkey UX promise.

The SDK closes the gap with two coordinated callbacks:

```
// At credential-change time, prompt for each registered passkey.
await tarn.account.changeCredentials(newEmail, newPw, {
  phrase: accountKey,
  passkeyTapHandler: async ({ credentialId, deviceLabel }) => {
    // Surface a UI: "Tap your passkey on '<deviceLabel>'."
    return await app.confirmPasskeyTap(deviceLabel ?? credentialId);
  },
});
```

If the account has registered passkeys and `passkeyTapHandler` is omitted, `changeCredentials` throws — silent stale credentials would be a worse failure than refusing to proceed. The handler returns `true` to proceed (SDK invokes WebAuthn for that credential, derives the wrapping key, attaches a wrapping to the new gen) or `false` to skip (credential ships

without a new-gen wrapping → becomes "stale"). If the user dismisses the WebAuthn prompt or the authenticator is unavailable, the credential is also marked stale; this is not an error.

Stale credentials are repaired transparently on next authenticate-with-passkey:

```
// On the next login with a stale passkey, supply a handler that
// returns BOTH the username and the password. The SDK derives the
// password KEK locally, unwraps the latest gen via password, re-wraps
// it under the passkey, and submits the repaired envelope. After this
// round-trip the credential is no longer stale.
await tarn.authenticateWithPasskey({
  stalePasskeyHandler: async () => ({
    username: await app.promptForUsername(),
    password: await app.promptForPassword(),
  }),
});
```

The handler returns both fields because passkey-only sessions don't have either cached on the client (the user has only ever tapped — never typed). Apps prompt for both, which is the same shape as a normal username+password login form. For Bookish-style apps where the username IS the user's email, this is a familiar pattern. If the handler returns `null`, the SDK aborts the repair and throws `StalePasskeyError`; if the returned object is missing either field, the SDK throws with a clear "must return { username, password }" message.

If `stalePasskeyHandler` is omitted and the credential is stale, the SDK throws `StalePasskeyError` (exported from the package) so the app can prompt re-registration via `tarn.passkeys.register()` from a password-authenticated session — the fallback recovery path.

The Settings UI can also surface staleness proactively: every entry returned by `tarn.passkeys.list()` carries a `stale` boolean. Render a "Refresh" button next to stale entries so users can repair them before they hit a stale credential at login time.

Synced passkeys (the dominant case). Apple iCloud Keychain and Google Password Manager share the same credential and PRF secret across all of a user's devices. A single re-tap on one device emits a wrapping that any of the user's other devices can derive themselves on next use — they never need to re-tap. For users who have registered multiple distinct credentials (e.g., iPhone Face ID *and* a YubiKey), each credential's staleness is independent.

Account + Session

```
// Rotate credentials (routine username/password change). Existing data stays
// decryptable.
// Pass the account key to extend the recovery factor to the new generation. If the
// account
// has any registered passkeys, also pass `passkeyTapHandler` – the SDK calls it once
// per
// credential to drive the re-tap that re-wraps the new generation under each passkey's
// PRF KEK. Without the handler, accounts with passkeys throw (see "What happens when
// you
// change your password" in the Passkeys section above for the full model).
await tarn.account.changeCredentials('new@example.com', 'new-password', {
  phrase,
  passkeyTapHandler: async ({ credentialId, deviceLabel }) =>
    await app.confirmPasskeyTap(deviceLabel ?? credentialId),
});

// Permanently delete. Tombstones credentials, clears server-side state, wipes local
// session.
await tarn.account.delete();

// Local session.
tarn.session.isLoggedIn();           // boolean – whether keys are loaded
await tarn.session.clear();           // log out: forget local session + wipe per-account
// caches

// Server-side session management (one row per device).
const devices = await tarn.session.listDevices();
// [{ sid, createdAt, lastSeenAt, deviceLabel, viaRecovery, isCurrent }, ...]

await tarn.session.revokeDevice(devices[1].sid); // log one device out
await tarn.session.revokeAllOthers();             // "log out everywhere else"
await tarn.session.revokeAll();                   // log out everywhere including here
```

Logout wipes the SDK's per-account local caches. `tarn.session.clear()` (and the other full sign-outs: `tarn.session.revokeAll()`, `tarn.account.delete()`) deletes, in addition to the persisted session blob, the SDK's per-account IndexedDB state for the account being signed out: the delta-sync cursors in `tarn-sync-cursors` and the ciphertext blobs in `tarn-blob-cache`. Apps do **not** need to delete those databases in their own logout handlers — and shouldn't have to know they exist. (Deleting them anyway is harmless; the SDK rebuilds both lazily.) The wipe is best-effort — a storage failure never breaks logout — and scoped to the current account, so other accounts that have used the same browser keep their caches and skip a full resync on their next login. Credential changes and account recovery do **not** wipe these caches: the account's `data_lookup_key` is stable across credential rotation, so the cursor and cached blobs remain valid, and wiping them would only force a needless resync.

Apps **should** attach a device label at login time — `tarn.login(username, password, { deviceLabel: 'Chrome on MacBook' })` — so users can identify their sessions in `listDevices()`. If omitted, `deviceLabel` is `null` and users only see the opaque `sid` and timestamps. Pick something the user will recognize: a User-Agent summary, a hostname, or a

name the user typed at signup.

Session persistence and the passkey-only session. Both password and passkey logins persist transparently: the SDK serializes the in-memory session under an origin-bound IndexedDB wrapping key, writes the ciphertext to `localStorage`, and resumes it on the next `TarnClient.create()` call. Passkey-only sessions persist and resume on the same code path — the resumed client returns `isLoggedIn() === true` and supports the same read + CRUD surface that the just-authenticated client does. The persisted blob hard-expires 7 days after creation with no refresh-on-use; on expiry the user re-authenticates (re-tap for passkey, retype for password). The password-side fields (`username`, `credentialLookupKey`, signing/sharing keypairs, `credentialEncryptionKey`, `recoveryFactorMeta`) are only present in the blob when the user signed in with a password — passkey-only blobs omit them, and the methods that need them throw a "requires a password-authenticated session" error rather than null-dereferencing.

Advanced (escape hatches)

Power-user surface for prototyping, debugging, and use cases the typed namespaces don't cover. Most apps never need this.

For an `arbitrary-type` that is NOT declared in your schema, these methods are pure pass-through — no validation, no auto-stamping, the caller manages every tag. When the `type` argument DOES match a declared collection, the SDK enforces the same invariants as the typed `Collection<T>` surface (schema validation + Eid auto-stamp); see the [Bulk writes](#) section below for details.

```
// Schema-less entry CRUD – bypasses the collection layer entirely.
await tarn.advanced.entries.create('arbitrary-type', { foo: 'bar' });
await tarn.advanced.entries.update(priorTxid, 'arbitrary-type', { foo: 'baz' });
await tarn.advanced.entries.delete(targetTxid, 'arbitrary-type');

// Raw blob fetch + decrypt by shareKey. Useful for one-off recipient flows.
const blob = await tarn.advanced.entries.fetchBlob(txid);
const data = await tarn.advanced.entries.decryptSharedBlob(blob, shareKey);

// Direct share-log access. Collection.share()/listShared() are the typed wrappers.
await tarn.advanced.shareLog.read(connection);
await tarn.advanced.shareLog.share(connection, contentId, txid, shareKey);
await tarn.advanced.shareLog.unshare(connection, contentId);
```

Bulk writes (`advanced.entries.batchCreate`)

Schema-less bulk-write up to 25 entries in a single request. Counts as **1 rate-limit hit** regardless of batch size. This is the escape hatch for `type` strings that are NOT on the schema — app-internal types (state caches, share-log helpers, etc.) where you manage tags yourself.

```
// App-internal type with no schema declaration. Caller manages tags.
const results = await tarn.advanced.entries.batchCreate('my-app-cache', records);
// results: [{ txid, shareKey }, ...] – same order as input.
```

For records that DO belong to a declared collection, prefer the typed path: `tarn.<collection>.batchCreate(records)` . That path validates every record against the schema (atomically — if any record fails, nothing is written) and stamps each entry with its Eid + SchemaV tags, so the batch surfaces normally through typed reads.

The advanced wrappers (`create` / `batchCreate` / `update`) enforce the same invariants when `type` matches a declared collection — schema validation runs, Eid + SchemaV are auto-stamped from each item's primaryKey, and a mixed-validity batch throws with input-indexed reasons before anything reaches the wire. Caller-supplied `Eid` / `SchemaV` tags are respected if present (no double-stamp), but a caller-supplied `Eid` that disagrees with the SDK-derived value throws `TarnSchemaError` — that's almost always a caller bug that would orphan the record from the typed read path. This closes the silent-orphan gap for defined collections regardless of which write surface the caller uses; the typed path is still recommended for ergonomics (full TS inference on the record shape).

Throws on `items.length === 0` **or** `items.length > 25` . Idempotent — a retry on the same input produces the same list of txids (one idempotency key per batch). Larger imports: chunk client-side and pace at ~1 batch per 36s to stay under the 100 writes/hour/account limit.

If you find yourself reaching for `advanced.*` for something the typed surface should cover, that's a signal to file an issue.

Reserved system types (why they bypass Eid / SchemaV)

A handful of entry `type` strings are **reserved** by Tarn itself and are managed by SDK primitives, not by app schemas. `defineSchema()` rejects any app collection that tries to claim one of these names:

Reserved type	Owned by	Purpose
<code>cred</code>	auth flow	credential / DEK-envelope mappings
<code>connection</code>	<code>tarn.connections.*</code>	friend connections
<code>share-log-state</code>	share-log (<code>tarn.advanced.shareLog.*</code>)	per-pair share-log entries
<code>share-inbox</code>	connection handshake	inbound connection-request material
<code>recovery-factor</code>	recovery / account-key	recovery key-derivation material
<code>app-config</code>	operator tooling	per-account app rules / config
<code>app-schema</code>	<code>publish-schema.mjs</code>	published schema documents

These records are **deliberately not Eid-addressed and carry no** `SchemaV` **tag**. They are protocol-internal structures with their own identity, ordering, and versioning rules — for example, share-log entries are addressed by per-pair stealth tags and sequence numbers, not by a derived Eid, and the credential envelope has its own generation chain. Stamping them with an app-collection `Eid / SchemaV` would be meaningless at best and would collide with the protocol's own addressing at worst. So if you inspect these types on the wire (or via the untyped surface) and notice they lack `Eid / SchemaV`, that is correct and intentional — they are not orphans and there is no auto-tagging to "fix." The reserved set is the single source of truth (`RESERVED_TYPE_NAMES`, exported from the SDK); a future system type must be added there and wired into the primitive that owns it.

App registration

Before users can register on your app, the app itself must be registered with Tarn. This is operator-level setup; it happens once per app, not per user.

1. Generate an app key pair

```
node tools/generate-app-key.mjs your-app-id
```

Prints the private key (hex), the public key, and a D1 seed SQL line. **Save the private key in a password manager** — it's printed once.

2. Register the app in Tarn

Run the printed D1 seed command:

```
# Local
cd api && npx wrangler d1 execute tarn-api --local --command "<printed SQL>"
# Production
cd api && npx wrangler d1 execute tarn-api --remote --command "<printed SQL>"
```

3. Set per-account rules

After a user registers, set their write rules:

```
node tools/set-rules.mjs \
  --api https://api.tarn.dev \
  --app your-app-id \
  --key <private key hex> \
  --dlk <user's data_lookup_key> \
  --plan free          # or annual, clear, deny, or --rules '<JSON>'
```

Plan / type	Effect
free	5 entries (app-wide), 100KB max per entry
annual	1000 entries (app-wide), expires in 1 year

Plan / type	Effect
<code>clear</code>	Empty rules (allow all)
<code>deny</code>	Deny all writes
<code>max_entries</code>	<code>{ limit, since?, app?, entry_type? }</code>
<code>max_bytes</code>	<code>{ limit }</code> per entry
<code>expires</code>	<code>{ at }</code> ISO 8601

Rules are AND logic; unknown rule types fail closed.

`max_entries` counts **resolved live entries** — what the user actually sees (tarn#65): superseded edit versions and deleted (tombstoned) entries don't count, and deleting an entry frees quota. The optional `entry_type` filter matches the entry's `Type` tag, which for schema-first SDK writes is the **collection name** (e.g. `books`) — the canned plans apply app-wide and don't set it.

4. Publish the schema

The schema is published to Arweave so the future "always access your data" recovery client can read it without going through the Tarn API. Run this once per release that bumps `schema.version`:

```
node tools/publish-schema.mjs \
  --api https://api.tarn.dev \
  --app your-app-id \
  --key <private key hex> \
  --schema ./schema.mjs
```

`--schema` accepts either a `.json` file (serialized schema) or a `.mjs` file with a default export equal to the schema. Re-running with the same `(app, version, schema)` is a safe no-op.

Examples

Four progressive examples live in `../examples/`:

- `01-hello-world` — register, write one record, list it.
- `02-crud` — full CRUD on two collections.
- `03-sharing` — invite-token handshake + share-with-all flow.
- `04-recovery` — register, capture the account-key string, simulate password loss + `recoverAccount`.

Each example is a standalone npm package linking to this client via `file:../../client`. See `examples/README.md` for setup.

TypeScript

The SDK is written in TypeScript and ships full `.d.ts` declarations. Schema-derived types flow through the client:

```
const tarn = await TarnClient.create({ schema: mySchema, /* ... */ });

await tarn.notes.create({
  noteId: 'n1',
  title: 'Foo',
  // titel: 'typo'    // ✗ TS error: unknown field
});

const note = await tarn.notes.get('n1');
//   ^? { noteId: string; title: string; body?: string; priority?: number; ... }
```

Plain JavaScript works too — the inference simply doesn't run. The runtime validators still enforce the schema at write time.

Error taxonomy

The SDK throws **typed error classes** for the failure modes apps realistically branch on — auth state, capability gaps, rate limits, schema/validation problems, and account-key tampering. Each is exported from the package root and sets a stable `.name`, so you can match with `instanceof` (preferred) or `err.name`. Everything else (network failures, unexpected 5xx, programmer errors) surfaces as a plain `Error` — the taxonomy is deliberately **bounded**, not exhaustive.

```
import {
  TarnNotAuthenticatedError,
  TarnPasskeyOnlyError,
  TarnRateLimitError,
  TarnCollectionError,
  TarnSchemaError,
  TarnSchemaVersionError,
  AccountKeyPinningError,
} from 'tarn-client';
```

Class	Thrown when	Typical app response
<code>TarnNotAuthenticatedError</code>	An authenticated method is called on a client that never logged in (no JWT, no signing keys), or whose JWT expired with no signing keys available to silently re-authenticate. Raised by the internal auth gate every authenticated method passes through.	Route the user to the sign-in screen.

Class	Thrown when	Typical app response
<code>TarnPasskeyOnlyError</code>	An operation that needs password-derived (master-key) state is invoked on a passkey-authenticated session — share-log read/write, connection handshake, <code>changeCredentials</code> , and other account-key/credential-rotation ops.	Prompt a step-up: "This needs a password — sign in with your password to continue."
<code>TarnRateLimitError</code>	Any HTTP 429 from the API. Carries <code>retryAfterSeconds</code> , <code>url</code> , <code>responseBody</code> , <code>status</code> . The SDK does not auto-retry 429 (see Rate limiting).	Surface "rate-limited, try again in N", schedule a retry past <code>retryAfterSeconds</code> .
<code>TarnCollectionError</code>	A typed collection operation violates an invariant — failed schema validation on <code>create</code> / <code>update</code> / <code>batchCreate</code> (with the failing indexes for batches), or an attempt to mutate an immutable primary key via <code>update</code> .	Treat as a caller/validation bug; show the field error.
<code>TarnSchemaError</code>	<code>defineSchema()</code> is given an invalid schema (bad field types, a collection claiming a reserved system type, etc.), or an advanced write supplies an <code>Eid</code> that disagrees with the SDK-derived value.	Fix the schema / call site — a programmer error, not a runtime condition.

Class	Thrown when	Typical app response
<code>TarnSchemaVersionError</code>	A read encounters an entry written under a newer <code>SchemaV</code> than this client knows how to migrate. The typed read layer catches this internally and skips-with-warning rather than surfacing it; it is exported for apps that drive version dispatch directly.	Prompt the user to update the app.
<code>AccountKeyPinningError</code>	<code>tarn.accountKey.view()</code> detects a tampered or mis-bound account-key wrap (the stored wrap doesn't bind to this account).	Surface a security warning — the stored key material is not trustworthy.

Note: `StalePasskeyError` is also raised by the passkey authentication path (a credential with no wrapping for the current generation), but it is surfaced through the passkey-auth flow's handler contract rather than re-exported from the package root — match it by `err.name === 'StalePasskeyError'` if you need to branch on it directly. See [Passkeys](#).

Rate limiting

The Tarn API enforces per-IP and per-endpoint rate limits. When a request is rate-limited the server responds with HTTP 429 and (usually) a `Retry-After` header.

The SDK does not auto-retry on 429. Retrying a rate-limit signal immediately just confirms the client is going too fast and amplifies the problem — one logical sync could otherwise burn 3 quota slots in a few seconds. Instead, the first 429 surfaces as a typed `TarnRateLimitError` so the app can decide what to do.

```
import { TarnRateLimitError } from 'tarn-client';

try {
  await tarn.notes.list();
} catch (err) {
  if (err instanceof TarnRateLimitError) {
    // err.retryAfterSeconds – number | null, parsed from the Retry-After header
    // err.responseBody      – raw body text from the 429 response (often JSON
    //                        with { error, retry_after })
    // err.url               – full request URL that was rate-limited
    const wait = err.retryAfterSeconds ?? 60;
    showToast(`Rate-limited – try again in ${wait}s`);
    scheduleRetryIn(wait * 1000);
  }
}
```

```

    return;
  }
  throw err;
}

```

Suggested handling: wait at least `retryAfterSeconds` before retrying. If `retryAfterSeconds` is `null`, fall back to a conservative default (60s is reasonable for most endpoints). The SDK deliberately does not enforce a wait or schedule retries on the app's behalf — that policy belongs in the app, where it can be coordinated with the UX (banner, toast, sync queue, etc.).

5xx and transient network errors still retry transparently inside the SDK with exponential backoff. Only 429 is special-cased.

Security model

- **Client-side encryption.** All data is AES-256-GCM encrypted before leaving the client. The server never sees plaintext.
- **Argon2id KDF.** Memory-hard (m=64 MiB, t=3, p=1) for password→master-key. Sub-keys derived via HKDF-Expand.
- **ECDSA P-256 auth.** Challenge-response signing. No passwords transmitted; server stores only the public key.
- **Per-content CEK + forward-secret DEK rotation.** Each blob is encrypted with its own random CEK, wrapped under a generation-indexed DEK chain. Credential changes append a fresh DEK so post-rotation writes are unreachable from old credentials.
- **Multi-factor DEK chain.** Each chain entry is wrapped twice — once under a password-derived KEK, once under an account-key-derived KEK. Either factor independently unwraps the chain.
- **Arweave permanence.** Data is stored permanently on Arweave. Encrypted blobs are publicly visible but unreadable without the key.

Publicly observable metadata

Tarn protects content end-to-end, but a few metadata properties remain visible:

- **Connection-request inbox volume + timing** is observable to anyone who knows a user's `share_pub`. Casual social use is fine; sensitive contexts should consider `share_discoverable: false`.
- **Account existence via discoverability lookup** — a `share_discoverable: true` account leaks "this username is a Tarn user" to anyone who runs the lookup.
- **Once connected, share-log traffic is unlinkable** — per-pair tags are stealth-addressed; an Arweave observer cannot extract the connection graph from the protocol alone.

See [TARN_PROTOCOL.md § Publicly observable metadata](#) for the full breakdown, and [TARN_PROTOCOL.md](#) for the wire-protocol spec.